UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
ESCUELA DE POSGRADO

PROYECTO EDA

Motor inteligente de comercio electrónico: pipeline con ramas

Versión 7 · Informe ampliado con evidencia QUICK

AUTOR(ES)
Grace Selenia, Moscosso Flores
Debora Elsa, Jeronimo Balcazar
Vanessa Elizabeth, Burbano Quintero
Miguel Angel,Valdivia Bambaren

DOCENTE
Mejia Fernandez, Oscar Rodolfo

CURSO
Estructura de Datos y Algoritmos

Lima – Perú
2026

Resumen ejecutivo

Se diseñó un pipeline de procesamiento de eventos de comercio electrónico para limpieza y priorización, almacenamiento e indexación, búsqueda aproximada, recomendación y análisis de relaciones entre productos. La solución combina estructuras clásicas (cola FIFO, heap, hash, LRU, búsqueda binaria), probabilísticas (Bloom y Count-Min Sketch) y avanzadas (MinHash+LSH, HNSW, grafos y B+Tree propio persistente).

La evidencia reproducible más reciente proviene de una ejecución con 100.000 eventos crudos del archivo 2019-Oct.csv; después de eliminar 17 duplicados se obtuvieron 99.983 eventos. La evaluación controlada de escalabilidad v7 cubre 1.000 y 10.000 eventos, no 1 ni 10 millones. Se conserva esta distinción para evitar extrapolaciones.

Principales hallazgos

En test (300 sesiones), el recomendador de grafo obtuvo HitRate@5 de 31,33 %, frente a 14,33 % de HNSW y 28,67 % del híbrido. MinHash+LSH logró Recall@5 medio de 97 % con 64 bandas en 20 consultas, a costa de más candidatos. HNSW alcanzó Recall@5 de 100 % en 30 consultas sobre 5.452 productos. El B+Tree estático persistente validó siete consultas de rango.

1. Problema, alcance y objetivos

Los eventos de navegación, carrito y compra llegan en grandes volúmenes y requieren acceso rápido, control de duplicados, recuperación de candidatos y análisis de afinidad. El reto académico es justificar la selección de estructuras en términos de complejidad, calidad de resultado, costo de memoria y persistencia.

Objetivo general

Construir y evaluar un pipeline modular sobre REES46 que procese eventos de e-commerce y permita consultas, recomendaciones y análisis de grafos, comparando aproximaciones contra referencias exactas.

Objetivos específicos

• Implementar las etapas de limpieza, ordenamiento, priorización e indexación.

• Contrastar estructuras probabilísticas y de vecinos aproximados contra métodos exactos.

• Evaluar recomendaciones sobre particiones separadas de validación y prueba.

• Implementar y verificar un índice B+Tree persistente propio.

• Medir escalabilidad, dispersión temporal y memoria con protocolos reproducibles.

Alcance verificable

El archivo fuente utilizado es 2019-Oct.csv. El modo quick procesa una muestra de 100.000 registros. Las cifras de desempeño corresponden a las muestras y parámetros consignados en los CSV adjuntos; no representan métricas de producción ni garantías de generalización.

2. Datos y preprocesamiento

Trazabilidad entre versiones: el informe v2 reportó resultados iniciales sobre una muestra y protocolo distintos. Los resultados v7 se consideran la evidencia principal; no se interpretan diferencias entre v2 y v7 como mejoras causales ni se comparan porcentajes sin igualar muestras, particiones y configuraciones.

El notebook carga eventos, aplica limpieza y deduplicación, y separa histórico y flujo de evaluación. La corrida reportó 100.000 eventos crudos y 99.983 limpios. La división operativa aproximada 60 % histórico y 40 % flujo facilita distinguir construcción de índices y evaluación de eventos posteriores.

Riesgos de validez: sesgo temporal, escasez de cointeracciones, dispersión del grafo y representatividad limitada del modo quick. Las comparaciones de métodos de recomendación deben mantenerse sobre las mismas sesiones y protocolo.

3. Arquitectura del pipeline

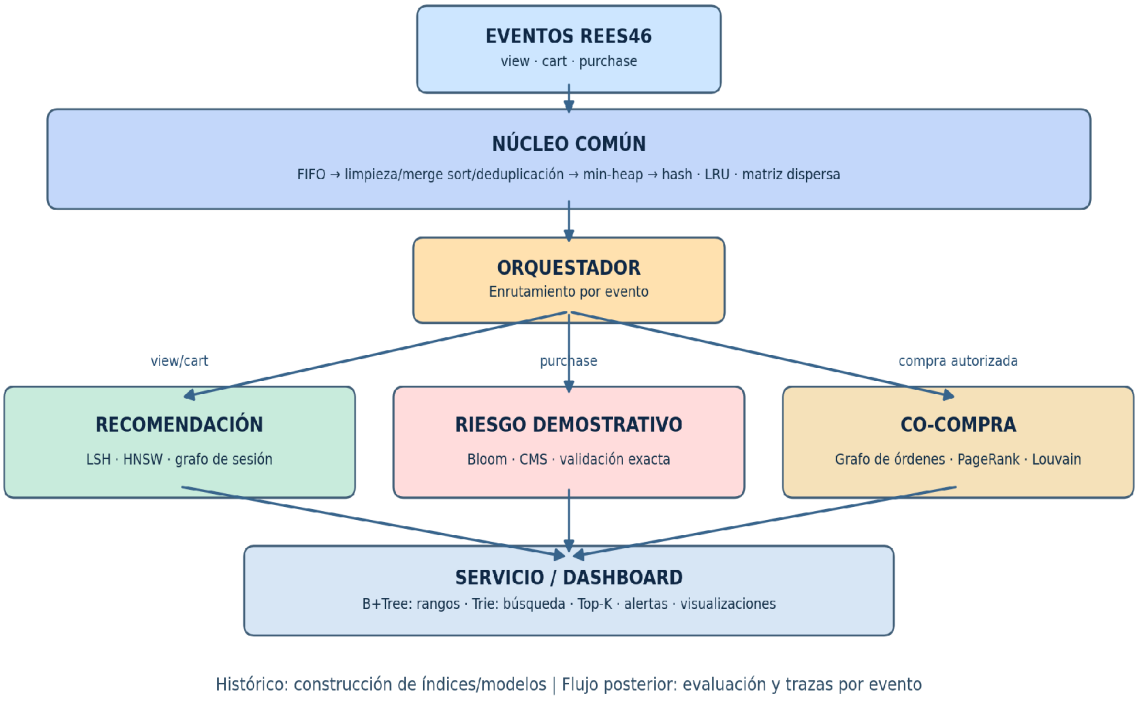


Figura 1. Arquitectura lógica del pipeline con núcleo común y ramas condicionales. Elaboración propia a partir del diseño v2, actualizado con la implementación v7.

La figura distingue la rama de recomendación (vistas y carritos), la evaluación demostrativa de compras y la actualización del grafo únicamente con compras autorizadas. El histórico alimenta los índices y modelos; el flujo posterior permite ejecutar consultas y registrar trazas. No representa una cadena que obligue a ejecutar todos los algoritmos en cada evento.

Etapa	Estructura / técnica	Propósito
0	FIFO / lista enlazada	Ingreso y orden de procesamiento
1	Ordenamiento y búsqueda binaria	Depuración y consultas sobre secuencias
2	Heap de prioridad	Atención de eventos prioritarios
3	Hash, matriz dispersa, LRU	Acceso directo, relaciones y caché
4	B+Tree persistente	Rangos de precio y almacenamiento en disco
5	KD-Tree y KNN exacto	Consultas espaciales/vectoriales de referencia
6	Bloom / Count-Min Sketch	Membresía y frecuencias aproximadas
7	MinHash+LSH / HNSW	Similitud y vecinos aproximados
8	Grafo, PageRank, Louvain	Afinidad y comunidades
9	Top-K y dashboard	Exposición de recomendaciones y métricas

Flujo lógico

Eventos → limpieza y deduplicación → separación histórico/flujo → construcción de índices y modelos → atención de consultas, recomendaciones y detección demostrativa de anomalías → visualización de resultados.

La clasificación demostrativa de fraude no constituye un sistema antifraude calibrado ni validado para decisiones reales. El notebook sirve como demostrador de estructuras y flujo.

4. Diseño de estructuras

Hash ofrece consultas promedio O(1) tras construir la tabla; un escaneo completo cuesta O(n). Heap permite extraer prioridades en O(log n), y la cola FIFO opera con inserción/extracción O(1) si mantiene extremos. Bloom y Count-Min Sketch sacrifican exactitud controlada para ahorrar recursos. LSH reduce candidatos potenciales, mientras HNSW aproxima KNN vectorial. PageRank explora importancia relativa de nodos y Louvain maximiza modularidad.

Estas complejidades son referencias teóricas bajo supuestos habituales; los tiempos empíricos dependen de implementación, tamaño, distribución y hardware.

5. Evaluación de recomendadores

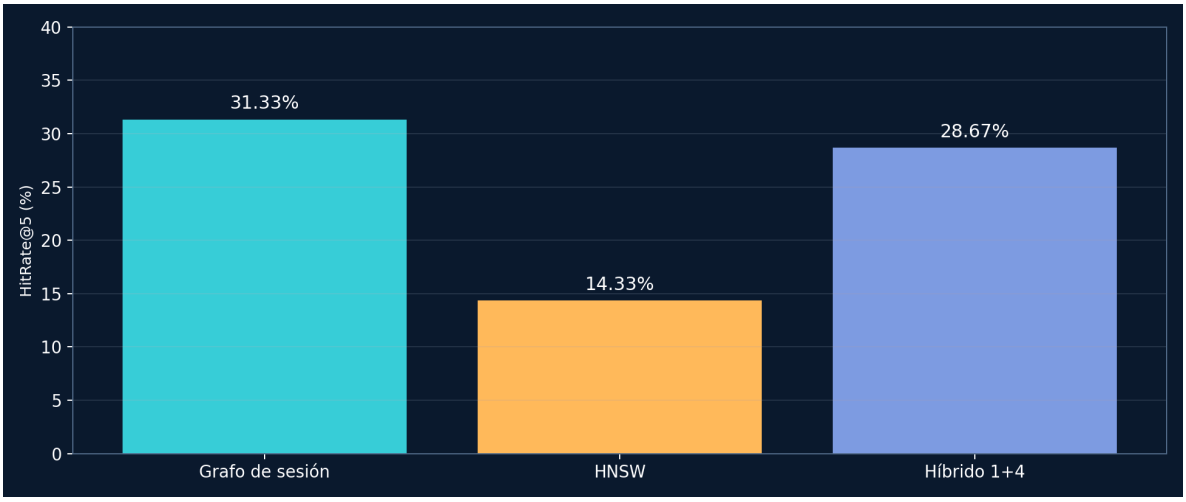


Figura 2. Comparación de recomendadores sobre sesiones de prueba. El grafo de sesión supera al híbrido en esta muestra.

Método	Sesiones	HitRate@5	Recall@5	Latencia mediana (ms)
Grafo de sesión	300	31.33%	16.09%	0.071
HNSW	300	14.33%	6.97%	0.815
Híbrido 1+4	300	28.67%	14.69%	1.007

El grafo de sesión supera a HNSW y al híbrido 1+4 en HitRate@5 y Recall@5 sobre las 300 sesiones de prueba. El híbrido no debe presentarse como mejora automática: bajo esta mezcla concreta fue inferior al grafo.

Separación validación / test

Método	HitRate@5 validación	HitRate@5 test
Grafo de sesión	21.33%	31.33%
HNSW	10.67%	14.33%
Híbrido 1+4	20.00%	28.67%

Las decisiones de selección de configuración deben realizarse con validación; el test se reserva para estimar desempeño final. Las cifras no deben interpretarse como una prueba causal de superioridad universal.

6. MinHash + LSH frente a Jaccard exacto

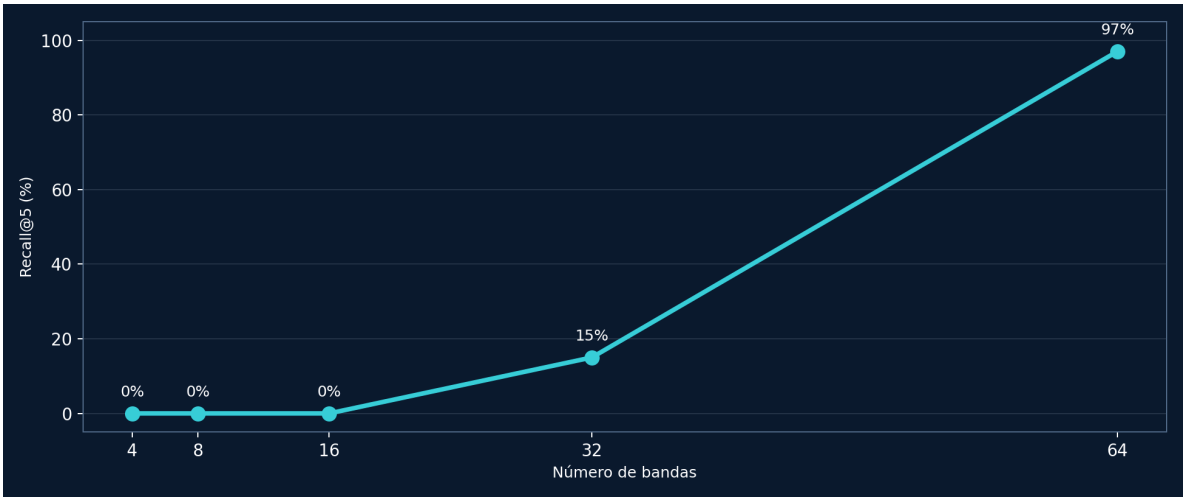


Figura 3. Sensibilidad de MinHash+LSH al número de bandas: recuperación y número de candidatos.

Bandas	Recall@5 medio	Candidatos promedio	Construcción (s)
4	0%	0.00	1.007
8	0%	0.00	1.005
16	0%	0.00	0.993
32	15%	1.35	1.051
64	97%	47.45	1.760

El barrido emplea 20 consultas evaluables. Con 16 bandas no se obtuvieron candidatos; con 64 bandas se recuperó 97 % de los cinco vecinos relevantes promedio, pero el número de candidatos ascendió a 47,45. Se evidencia el compromiso entre recuperación y costo de filtrado. El tiempo de construcción debe considerarse aparte del tiempo de consulta.

7. HNSW y comunidades

El experimento HNSW evaluó 30 consultas sobre un catálogo de 5,452 productos, con k=5. Recall@5 promedio: 100%.

Método	Tiempo promedio por consulta (ms)
KNN exacto	0.664
HNSW	0.079

HNSW fue más rápido en esta ejecución y coincidió con el top-5 exacto en las consultas seleccionadas. La recuperación perfecta en esta muestra no implica exactitud universal.

Algoritmo	Modularidad media	Tiempo medio (ms)	Estabilidad ARI
label_propagation	0.883	6.23	0.883
louvain	0.911	133.37	0.839

Se compararon tres semillas sobre un grafo de 840 nodos y 167 aristas. Louvain alcanzó mayor modularidad; Label Propagation resultó más rápido y ligeramente más estable según ARI. La gran dispersión del grafo y el alto número de comunidades exigen prudencia al traducir las comunidades a segmentos comerciales.

8. Índice B+Tree persistente

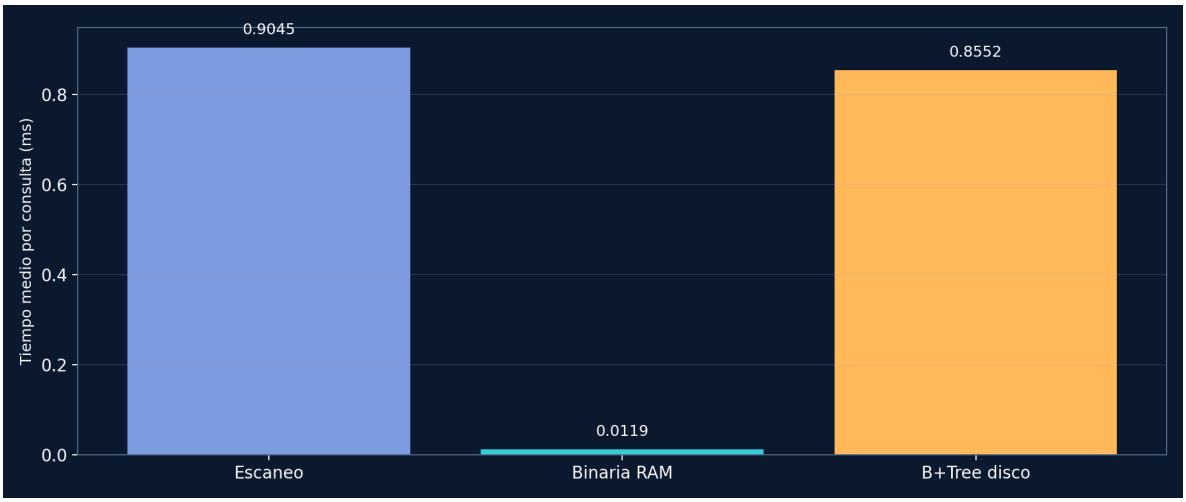


Figura 4. Latencias de consulta por rango: escaneo, búsqueda binaria en RAM e índice persistente.

Se implementó un B+Tree propio, estático y construido por carga masiva, con páginas persistentes y hojas enlazadas. El índice se cierra y reabre para comprobar persistencia y ejecuta búsquedas por rango. El inventario reporta 20.621 productos y altura 3.

Las 7 consultas de rango del archivo de validación presentan coincidencia exacta: 7 de 7.

Productos	Escaneo (ms)	Bisect RAM (ms)	B+Tree disco (ms)
1000	0.078	0.004	0.126
10000	0.905	0.012	0.855

La búsqueda binaria en RAM fue más rápida en estos tamaños. El aporte del B+Tree es la persistencia y organización de rangos; no se acreditan inserciones ni eliminaciones incrementales.

9. Benchmark controlado y escalabilidad

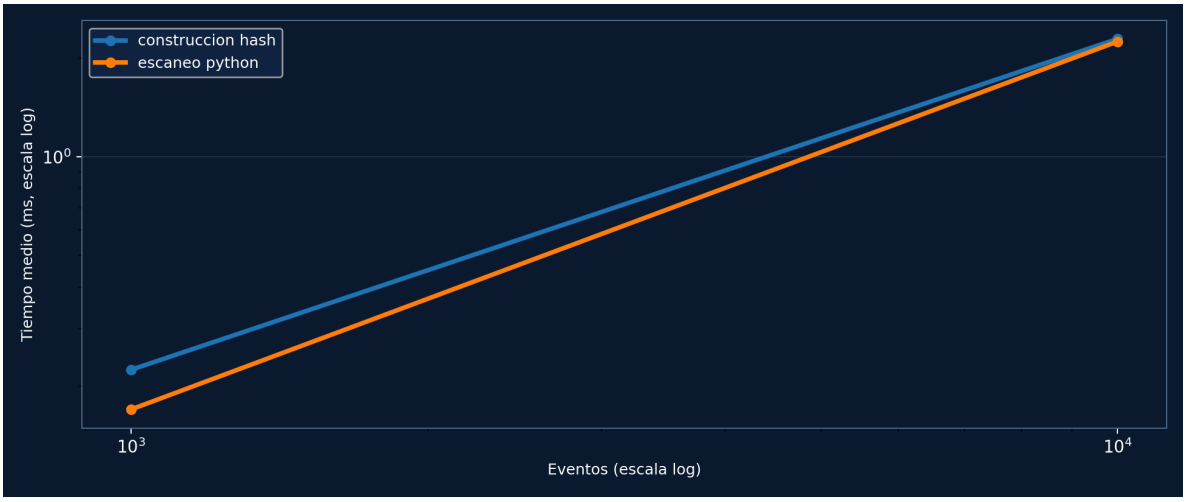


Figura 5. Escalabilidad observada en modo QUICK; solo 1.000 y 10.000 eventos medidos.

Operación	1.000 eventos (ms)	10.000 eventos (ms)
escaneo_python	0.169740	2.243748
consulta_hash_preconstruido	0.000283	0.000269
construccion_hash	0.224262	2.286165

Protocolo: semilla 42, plataforma Linux x86_64, 15 repeticiones por operación y tamaño, con múltiples llamadas por lote para mejorar la resolución temporal. Los CSV registran media, desviación estándar, checksum y número de llamadas.

Las consultas hash miden acceso a una tabla ya construida, mientras que la construcción se mide por separado. Por ello, no se debe comparar el tiempo de consulta aislado con el costo total de indexación sin declarar esta diferencia.

Eventos	Pico de memoria rastreada (MB)	Claves únicas
1000	0.049	534
10000	0.252	4150

La memoria corresponde a asignaciones rastreadas por tracemalloc; no equivale al máximo RSS del proceso ni a la memoria total consumida por NumPy o librerías nativas.

Escalas todavía pendientes

La evidencia v7 quick solo contiene mediciones de 1.000 y 10.000 eventos. Los objetivos de 100.000, 1.000.000 y 10.000.000 eventos permanecen pendientes de validación mediante la ejecución FULL. No se muestran extrapolaciones como resultados observados.

10. Discusión y limitaciones

Los resultados favorecen al grafo de sesión para la recomendación en esta muestra, muestran la sensibilidad de LSH a la configuración de bandas y verifican que HNSW puede acelerar KNN sin pérdida observada de recall en 30 consultas. La persistencia del B+Tree fue validada funcionalmente, aunque su consulta no superó la búsqueda binaria en memoria.

Las limitaciones incluyen: tamaño acotado del modo quick; poca densidad del grafo; selección limitada de consultas; posible variabilidad por cachés y entorno compartido; B+Tree sin actualizaciones; y medición parcial de memoria. El sistema de fraude es ilustrativo, no un detector evaluado con etiquetas de fraude.

Amenazas a la validez

Validez interna: las operaciones comparadas no siempre tienen idénticos costos de preparación. Validez externa: un mes y una muestra de REES46 pueden no representar otros catálogos. Validez de constructo: HitRate@5, Recall@5, modularidad y ARI son proxies técnicos y no ingresos, satisfacción ni impacto comercial.

11. Conclusiones

11.1. Viabilidad de una arquitectura basada en estructuras de datos especializadas

El desarrollo del proyecto permitió implementar un pipeline de procesamiento de eventos de comercio electrónico que integra estructuras de datos clásicas, probabilísticas y avanzadas. Los resultados evidencian que no existe una estructura universalmente superior: su conveniencia depende de las características del problema, del volumen de información, del tipo de consulta y de las restricciones de memoria y persistencia.

En consecuencia, la principal contribución arquitectónica del proyecto es demostrar la importancia de seleccionar estructuras de datos en función de los patrones de acceso y los requerimientos operativos, en lugar de priorizar únicamente su sofisticación algorítmica.

11.2. Desempeño de los algoritmos de recomendación

En la evaluación realizada sobre el conjunto de prueba, el recomendador basado en grafos de sesión obtuvo un HitRate@5 de 31,33 %, superando a las alternativas basadas exclusivamente en HNSW y en la combinación híbrida evaluada.

Este resultado sugiere que, para las interacciones disponibles en la muestra, las relaciones observadas entre productos dentro de una sesión proporcionaron señales relevantes para anticipar futuras interacciones.

Asimismo, evidencia que incorporar un componente adicional a un sistema de recomendación no garantiza una mejora de desempeño. La combinación de algoritmos debe justificarse mediante validación empírica y no únicamente por su complementariedad teórica.

11.3. Compromiso entre precisión y eficiencia en la búsqueda aproximada

Los experimentos con MinHash y Locality-Sensitive Hashing evidenciaron una sensibilidad importante a la configuración de parámetros. Mientras que la configuración inicial de 16 bandas no recuperó candidatos, la configuración de 64 bandas alcanzó un Recall@5 promedio de 97 % en las 20 consultas evaluadas.

Sin embargo, este incremento estuvo acompañado de un mayor número de candidatos por consulta. Por tanto, mejorar la recuperación no implica necesariamente optimizar el costo computacional total.

De manera complementaria, HNSW obtuvo un Recall@5 de 100 % frente a KNN exacto en las 30 consultas seleccionadas, con una menor latencia promedio. Estos resultados respaldan el potencial de los métodos aproximados, aunque su comportamiento debe seguir evaluándose con catálogos más grandes y distribuciones diferentes.

11.4. Persistencia y rendimiento del índice B+Tree

La implementación de un B+Tree propio permitió validar consultas por rango y persistencia del índice mediante almacenamiento en disco, cierre y reapertura.

No obstante, los benchmarks mostraron que la búsqueda binaria sobre datos residentes en memoria fue más rápida para los tamaños evaluados. Este resultado no invalida el uso del B+Tree, sino que permite distinguir entre dos objetivos de diseño: minimizar la latencia de consulta en memoria y mantener una estructura de indexación persistente.

La implementación actual corresponde a un índice estático construido mediante carga masiva. Su evolución hacia inserciones y eliminaciones incrementales constituye una oportunidad de mejora.

11.5. Interpretación de los resultados experimentales y escalabilidad

Los benchmarks controlados permitieron observar que el costo del escaneo completo y de la construcción de una tabla hash aumenta con el volumen de eventos, mientras que el tiempo de consulta de un hash previamente construido permaneció relativamente estable en las escalas evaluadas.

Estos resultados son consistentes con el comportamiento esperado de las estructuras analizadas, pero no constituyen por sí solos una demostración empírica de sus complejidades asintóticas.

La evidencia disponible corresponde a ejecuciones sobre muestras acotadas. La evaluación ampliada de hasta diez millones de eventos permanece sujeta a los resultados de la ejecución FULL, por lo que no se formulan conclusiones definitivas sobre ese volumen.

11.6. Implicaciones para el diseño de soluciones de datos

Desde una perspectiva de ingeniería de datos, los experimentos demuestran que las decisiones arquitectónicas deben considerar conjuntamente calidad de respuesta, costo computacional, memoria, persistencia y mantenibilidad.

En aplicaciones de comercio electrónico, estas decisiones pueden influir en la capacidad para responder consultas de productos, recuperar recomendaciones relevantes y analizar relaciones entre usuarios y artículos.

Sin embargo, el desempeño técnico observado no debe interpretarse directamente como impacto comercial. Para establecer beneficios sobre conversión, ventas o experiencia del cliente serían necesarias evaluaciones adicionales con métricas de negocio.

11.7. Conclusión general

El proyecto demuestra la factibilidad de integrar múltiples estructuras de datos y algoritmos en un pipeline funcional de análisis de eventos de comercio electrónico, sustentando su comportamiento mediante comparaciones experimentales.

El aprendizaje central es que **la eficiencia de una solución no depende exclusivamente de utilizar algoritmos avanzados, sino de seleccionar, configurar y combinar adecuadamente las estructuras de datos según el problema que se busca resolver**.

Los resultados obtenidos constituyen una base técnica para futuras optimizaciones, particularmente en escalabilidad, actualización de índices, calidad de recomendaciones y validación bajo condiciones más próximas a un entorno productivo.

12. Fundamentación de las decisiones de arquitectura

Esta sección profundiza la justificación técnica del sistema y distingue las propiedades teóricas de los resultados observados. La arquitectura no se concibe como una secuencia de algoritmos independientes, sino como una combinación de estructuras que resuelve patrones de acceso distintos: procesamiento temporal, consulta por clave, filtrado aproximado, búsqueda vectorial y explotación de relaciones.

12.1. Patrón de acceso y elección de estructura

Necesidad	Alternativa y razón	Costo / compromiso
Orden de llegada	FIFO preserva secuencia de atención	O(1) encolar/desencolar con punteros a extremos; no ordena por prioridad
Eventos urgentes	Heap separa prioridad del orden temporal	O(log n) inserción y extracción; exige criterio de prioridad explícito
Acceso por identificador	Hash para claves exactas	O(1) promedio; O(n) en colisiones adversas y memoria adicional
Rangos de precios	B+Tree en páginas persistentes	O(log_B n + k/B) E/S idealizada; páginas, caché y escritura condicionan latencia
Pertenencia frecuente	Bloom como filtro preliminar	Falsos positivos posibles; no falsos negativos bajo inserción estándar
Conteos aproximados	Count-Min Sketch	Sobreestimación por colisiones; depende de ancho y profundidad
Similitud entre conjuntos	MinHash+LSH como generador de candidatos	Recall sensible a bandas y similitud de datos
Vecinos vectoriales	HNSW para KNN aproximado	Trade-off entre recall, memoria y ef_search
Afinidad producto-producto	Grafo y PageRank/comunidades	Dependencia fuerte de densidad y definición de arista

12.2. Costos amortizados y distinción entre construir y consultar

Un error de comparación frecuente consiste en contraponer el tiempo de una consulta hash sobre un índice ya construido con el tiempo de escanear un arreglo sin incluir el costo inicial de construcción. Si se realizan q consultas sobre n eventos, el costo total aproximado de la alternativa hash es $T_{hash}(n,q)=T_{construcción}(n)+q\cdot T_{consulta_hash}$; para escaneo, $T_{scan}(n,q)=q\cdot T_{scan}(n)$. El punto de equilibrio depende de q, del patrón de claves y de la memoria disponible. Por ello se reportan por separado construcción y consulta.

12.3. Persistencia y modelo de E/S

El B+Tree construido por carga masiva separa el almacenamiento del ciclo de vida de Python. El árbol persistido permite cerrar el proceso, volver a abrir el archivo y ejecutar rangos sin reconstruir toda la estructura. La altura 3 observada describe el índice construido, pero no implica por sí sola superioridad de latencia frente a una lista ordenada residente en RAM. La comparación justa debe indicar dónde reside cada estructura, si el sistema operativo mantiene páginas en caché y si se mide tiempo de apertura, construcción o solo consulta.

13. Diseño experimental y reproducibilidad

13.1. Preguntas e hipótesis de evaluación

Pregunta	Hipótesis contrastable	Métrica

¿Filtrar con LSH reduce el universo de búsqueda?	Menos candidatos que Jaccard exhaustivo, con pérdida acotada de recall	Recall@5, candidatos, tiempo de construcción y consulta
¿HNSW acelera KNN?	Menor latencia con recall suficientemente alto	Recall@5, ms/consulta, n catálogo
¿Qué comunidad representa mejor la red?	Louvain mayor modularidad; Label Propagation menor costo	Modularidad, ARI entre semillas, tiempo
¿Qué recomendador generaliza mejor?	Diferencias medibles entre grafo, HNSW e híbrido	HitRate@5, Recall@5 y latencia sobre test
¿Qué aporta B+Tree?	Persistencia y rango correctos, no necesariamente menor latencia	Exactitud, tiempo, altura, tamaño archivo
¿Cómo escala el acceso a eventos?	Escaneo crece con n; consulta hash estable en promedio	Media, desviación, memoria rastreada y log-log

13.2. Protocolo de medición

La ejecución v7 quick documenta semilla 42, 15 repeticiones por operación y tamaño para el benchmark controlado, y tamaños de 1.000 y 10.000 eventos. El uso de llamadas por lote reduce la dominancia de la resolución del reloj en operaciones muy rápidas. La desviación estándar cuantifica dispersión observada, pero no constituye por sí sola un intervalo de confianza ni garantiza independencia entre corridas.

Para el experimento FULL se propone conservar: fecha y versión del notebook, plataforma y RAM disponible, n real procesado, número de consultas, tiempo de calentamiento, número de repeticiones, criterios de descarte y capturas de la ejecución. Si el proceso termina por memoria insuficiente, debe registrarse como límite observado, no sustituirse por valores estimados.

13.3. Definiciones formales de métricas

Recall@K para recuperación aproximada = |TopK_aproximado ∩ TopK_exacto| / K cuando la referencia exacta contiene K elementos. HitRate@K para recomendación = promedio de 1 si al menos un ítem objetivo está entre las K recomendaciones, y 0 en caso contrario. Recall@K de recomendación = promedio de ítems objetivo recuperados / ítems objetivo por sesión; su denominador puede diferir de K. Modularidad Q contrasta el peso intra-comunidad con el esperado bajo un modelo nulo de grados. ARI mide acuerdo entre particiones corrigiendo por azar. Estas métricas evalúan propiedades diferentes y no son intercambiables.

13.4. Control de fuga de información

La división histórico/flujo y la separación validación/test son decisiones metodológicas esenciales. Una recomendación basada en relaciones construidas con eventos posteriores a la consulta produciría fuga temporal. Asimismo, elegir la mezcla híbrida después de observar el resultado en test sobreajustaría el conjunto de evaluación. La configuración se debe fijar con validación y reportar el test una sola vez por versión congelada.

14. Interpretación crítica de los experimentos

14.1. Por qué 64 bandas mejoraron LSH

Para MinHash con b bandas y r filas por banda, la probabilidad idealizada de generar al menos una coincidencia de banda para dos conjuntos con similitud Jaccard s es P(candidato)=1−(1−s^r)^b, asumiendo el esquema clásico de firmas y bandas. Con 64 permutaciones, pasar de 16×4 a 64×1 incrementa fuertemente la probabilidad de admitir pares de similitud moderada, pero también aumenta falsos candidatos. El Recall@5 de 97 % con 64 bandas, por tanto, no equivale a demostrar mayor precisión ni menor tiempo extremo a extremo; expresa mayor cobertura del top-K exacto en 20 consultas.

14.2. HNSW: rapidez sin garantía de exactitud

HNSW organiza un grafo navegable multinivel y explora vecinos candidatos sin comparar necesariamente todos los vectores. Su ventaja observada (aproximadamente 8,8× menor tiempo medio en el experimento v5) depende del catálogo, dimensionalidad, parámetros de construcción y búsqueda y del calentamiento del índice. Recall@5=100 % en 30 consultas significa coincidencia en esa muestra, no que el algoritmo haya dejado de ser aproximado.

14.3. Comunidades en un grafo disperso

El grafo del experimento tiene 840 nodos y 167 aristas, por lo que su densidad no dirigida aproximada es $2m/[n(n-1)] \approx 0{,}000474$. En redes así, numerosas componentes o grupos pequeños pueden inflar la modularidad sin producir segmentos útiles para negocio. Se recomienda complementar la comparación con distribución de tamaños de comunidades, porcentaje de nodos aislados y estabilidad frente a cambios del umbral de copra.

14.4. Recomendación: complejidad no equivale a utilidad

El grafo de sesión superó al híbrido ensayado en el conjunto de prueba. Esta observación es importante: combinar dos técnicas sofisticadas no garantiza mejorar la métrica objetivo. Una explicación plausible —todavía no verificada causalmente— es que la coocurrencia reciente entre productos represente mejor el siguiente comportamiento de la sesión que las características vectoriales disponibles. Para contrastarla harían falta ablaciones, distintos pesos híbridos seleccionados en validación y análisis por longitud de sesión.

14.5. Interpretación de la escalabilidad

Los resultados de 1.000 y 10.000 eventos son consistentes con la teoría: crecen los tiempos de escaneo y construcción de hash, mientras la consulta de hash preconstruido se mantiene pequeña. Sin embargo, dos puntos no permiten ajustar con confianza exponentes de complejidad. El gráfico log-log es una visualización exploratoria, no una demostración matemática de O(n) u O(1). Las mediciones FULL permitirán evaluar cambios de régimen por caché, asignación de memoria y presión del recolector de basura.

15. Riesgos, ética y aplicabilidad

La detección de fraude del pipeline es demostrativa y no debe automatizar bloqueos reales sin etiquetas, evaluación de falsos positivos/falsos negativos, calibración, auditoría de sesgos y mecanismos de revisión humana. En recomendaciones, las métricas offline no capturan diversidad, novedad, sesgo de popularidad ni impacto comercial. La separación entre desempeño algorítmico y decisión de negocio es una condición de uso responsable.

La reproducibilidad exige proteger los datos de eventos y no publicar identificadores personales innecesarios. En una implementación productiva se necesitarían retención limitada, control de accesos, monitoreo de deriva, reconstrucción incremental de índices y evaluación A/B.

16. Referencias conceptuales para ampliar la bibliografía

Las referencias siguientes sustentan el marco conceptual. Los resultados cuantitativos corresponden a los archivos de evidencia del proyecto y deben acompañarse del notebook, CSV y versión de ejecución. La ejecución FULL permanece pendiente de validación y no se reporta como concluida.

Estas obras se sugieren como sustento teórico para la versión académica; las cifras experimentales de este informe proceden exclusivamente de los CSV propios y no de estas publicaciones.

• Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press.

• Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, 13(7), 422–426.

• Cormode, G., & Muthukrishnan, S. (2005). An improved data stream summary: The count-min sketch and its applications. Journal of Algorithms, 55(1), 58–75.

• Broder, A. Z. (1997). On the resemblance and containment of documents. Compression and Complexity of Sequences.

• Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE TPAMI, 42(4), 824–836.

• Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. Journal of Statistical Mechanics, P10008.

• Page, L., Brin, S., Motwani, R., & Winograd, T. (1999). The PageRank citation ranking: Bringing order to the Web. Stanford technical report.